ERGALICS STUDIO · 技术文档

流体双向耦合求解器容器化部署手册

让每一次科学计算，都发生在离思想最近的地方

在任意装有 Docker 的主机上一条命令运行双向耦合可运行原型，并在容器内复现 26 项验证断言与基准。含一键编排、CLI 复现、生产静态模式与故障排查。

官网
snowleopard-io.github.io/ErgalicsStudio

GitHub
github.com/SnowLeopard-io/ErgalicsStudio

Gitee
gitee.com/cnt-code/ergalics-studio

本篇章节

一 1. 交付物清单
二 2. 前置条件
三 3. 快速开始
四 4. 常用运维
五 5. 与本项目交付体系的
六 6. 故障排查

独立专题文档 · Standalone Note

目 录

1. 交付物清单 **03**

2. 前置条件 **04**

3. 快速开始 **04**

3.1 一键启动可运行原型 04

3.2 容器内复现验证算例（CLI） 05

3.3 可选：生产静态模式（预构建 + nginx） 05

4. 常用运维 **06**

5. 与本项目交付体系的对应 **06**

6. 故障排查 **06**

配套交付物：`流体双向耦合求解器` 插件的**容器化部署包**。 本手册说明如何在任意装有 Docker 的机器上用一条命令跑起**可运行原型**，并在容器内复现全套验证算例。 涉及赛题交付要求（4b）"以容器化部署包形式提供可执行程序"。

配套交付物：`流体双向耦合求解器` 插件的**容器化部署包**。 本手册说明如何在任意装有 Docker 的机器上用一条命令跑起**可运行原型**，并在容器内复现全套验证算例。 涉及赛题交付要求（4b）"以容器化部署包形式提供可执行程序"。

---

1. 交付物清单

容器化部署包位于项目根目录，共 5 个文件：

文件	作用
`Dockerfile`	可运行原型镜像：Vite dev server 托管完整工作台
`docker/cfd.Dockerfile`	CLI 复现镜像：同一求解内核（纯 NumPy）+ 验证套件
`docker-compose.yml`	一键编排 `studio`（浏览器演示）与 `cfd-verify`（CLI 复现）
`.dockerignore`	构建上下文排除项（node_modules / 构建产物 / Rust 工具链）
本手册	部署、运行、复现与故障排查说明

与在线体验的关系：本项目同时提供 GitHub Pages 在线体验 （`https://snowleopard-io.github.io/ErgalicsStudio/`）。容器化部署用于需要**本地、离线、可审计**运行原型的场景，两者共用同一份代码与结果。

---

2. 前置条件

前置条件	要求
Docker	含 Compose 插件，Docker Desktop / 对应 Linux 包均可，`docker --version` 与 `docker compose version` 均可用
网络	可访问 Docker Hub（拉取 `node:20-alpine` / `python:3.11-slim`）与 npm registry（首次构建安装前端依赖）
内存	建议可用 ≥ 4 GB（浏览器端 Pyodide 运行 3D N–S 求解）；网格 10^3 通常仅需 ~1 GB
磁盘	余量 ≥ 2 GB（构建缓存）

---

3. 快速开始

3.1 一键启动可运行原型

```
# 在项目根目录执行
docker compose up -d --build studio
```

首次构建会安装前端依赖，需 2–8 分钟；之后启动为秒级。

浏览器访问 `http://localhost:5173`，进入左侧插件栏 `最新上架 → 流体双向耦合求解器`：

1. 选择算例（Case A 壅塞 / Case B 毫秒阀控 / Case C 能量通道 / Case D 亚声速 / 自定义）；
2. 点 **Run Coupling** 运行 1D 管网 ↔ 3D N–S 双向耦合；
3. 查看 1D 流量 / 阀门开度 / 背压时间序列，以及 **3D 场动态回放**（温度 / 速度幅值双通道、时间轴拖动播放）。

> CFD 求解器为纯 Python + Pyodide，**完全在浏览器端执行**，无需任何后端 Python。 原型容器只负责托管工作台前端，因此 `studio` 镜像无需 Rust/WASM 工具链，构建快、体积可控。

3.2 容器内复现验证算例（CLI）

```
# 完整验证套件（Case A/B/C/D + 亚声速扫描 + 权衡 + 最小周期 + 归因，共 26 项断言）
docker compose run --rm cfd-verify

# 指定数值求解入口（输出 JSON 到 stdout）
docker compose run --rm cfd-verify python -m fluid_cfd.driver verify

# 生成精度-效率基准 JSON（显式写回 /fluid；容器为一次性执行，如需保留可挂载卷或 docker cp）
docker compose run --rm cfd-verify python benchmarks/bench_coupling.py /fluid/fluid-cfd-results.json

# 单算例耦合（自定义 payload，见 config.example.json；CLI 子命令为 solve，后接 JSON 字符串）
docker compose run --rm cfd-verify python -m fluid_cfd.driver solve '{"case": "a"}'
```

以上与浏览器插件共用**同一份 `fluid_cfd` 源码**，结果逐字节一致。

3.3 可选：生产静态模式（预构建 + nginx）

`studio` 默认以 dev server 运行，够演示且构建快。若需要更贴近"生产部署"的静态形态， 可用本地工具链预构建后交给任意静态服务器（本仓库用于 GitHub Pages 的构建命令）：

```
# 需要 Rust + wasm32-unknown-unknown 目标 + wasm-bindgen-cli 0.2.127
npm ci && npm run build          # → dist/
# 用 nginx/http-server/vite preview 托管 dist/ 即可
```

> 该路径与部署 CI（`.github/workflows/deploy.yml`）一致，通常是"线上发布"的选择； 本地评审演示使用 §3.1 的 dev 容器即可，无需安装 Rust。

---

4. 常用运维

需求	命令
查看服务状态	`docker compose ps`
日志（前端 dev server）	`docker compose logs -f studio`
停止并移除	`docker compose down`
重启原型	`docker compose restart studio`
改端口（如 8080）	编辑 `docker-compose.yml` 中 `ports: ["5173:5173"]` → `["8080:5173"]`，再 `docker compose up -d`

---

5. 与本项目交付体系的对应

赛题交付要求	本包对应
可运行原型（4b）	`studio` 服务：浏览器内直接运行双向耦合 + 3D 动态回放
可复现材料（4d）	`cfd-verify` 服务：同一内核 26 项断言 + 基准生成
部署便捷、脚本化	`docker compose up -d --build studio` 一条命令
与方案/算例报告一致	结果口径与 `docs/technical/10-流体双向耦合求解器.md` 完全统一

---

6. 故障排查

现象	处理
`studio` 反复重启，端口被占	关闭占用 5173 的进程，或按 §4 改端口
浏览器打开白屏 / 无法加载插件	确认访问的是 `http://localhost:5173` 且日志无编译错误；`docker compose logs -f studio`

现象	处理
`cfd-verify` 网络拉取 numpy 慢	首次构建需下载 numpy wheel；后续有层缓存
3D 演示卡顿	网格默认 10^3；在插件「矩阵维度」选用更小网格减小 Pyodide 运算量
构建需 Rust	仅 §3.3 生产静态模式需要；§3.1 默认容器不需要

ERGALICS STUDIO · 宗旨

让每一次科学计算，

都发生在离思想最近的地方

Ergalics Studio 把复杂度收进浏览器：安装、配置与数据搬运都不该出现在科研的路上。打开即可计算，数据留在本机，四种工作模式共享同一份数据——一张图、一条流程与一行代码，都是同一次工作的不同表达。

工具越安静，思想就越清楚。我们做的每一处取舍，都为了把注意力还给问题本身。

作者 SnowLeopard-io github.com/SnowLeopard-io

官网 snowleopard-io.github.io/ErgalicsStudio
GitHub github.com/SnowLeopard-io/ErgalicsStudio
Gitee gitee.com/cnt-code/ergalics-studio

独立专题文档 · Standalone Note